

Evaluation of IPC through Linux Pipes

Patrick DeCabooter and Shaw Liu

1 Introduction

2 Clock Precision Evaluation

We tested two types of timers for our experiments. The first timer, provided by the header `time.h`, is `clock_gettime`. We chose `CLOCK_MONOTONIC` as the clock to use for `clock_gettime` due to it having guaranteed monotonicity and being system-wide, this provides nanosecond precision. The second timer, provided by the header `sys/time.h`, is `gettimeofday`. `gettimeofday` finds the time by measuring the clock cycles since the last kernel epoch, this provides microsecond precision. `gettimeofday` is a much older clock. By the time `cpu-cycles` were able to measure nanoseconds, `gettimeofday`'s interface was already locked-in. `clock_gettime` was introduced as the api to replace `gettimeofday`. In order to have similar units, we scaled `gettimeofday` up to nanoseconds instead of scaling `clock_gettime` down to microseconds. We chose to do it this way since it would take advantage of `clock_gettime`'s increased accuracy and give us more intuitive numbers for figures.

To evaluate the accuracy of these two timers, we decided to measure accuracy against `sleep` from the header `unistd.h` and measure the smallest measurable time. To measure against `sleep`, we started the timer, called `sleep(5)`, then measured the final time to get the difference. We did this five times in a loop for each timer, and took the minimum result. To measure the smallest measurable time, we had to compromise to instead find the smallest measurable time for `gettimeofday` since it measures in microseconds instead of nanoseconds. To make a one microsecond delay, we summed the numbers from 0 to 99 in a loop. We did this this 10 times and took the minimum result.

We used minimums as our metric for both tests since we wanted to find the highest fidelity possible if our machine was running perfectly.

Results

3 Pipe Latency Evaluation

Determining the latency of Pipes is crucial for performant IPC. When data moves between processes in a Pipe, it is not as simple as the other process reading it out of virtual memory. First, the data must be sent by `write` and cross the system call barrier to enter kernel-space memory and use the Pipe writing file descriptor as an address. Then, the reader must call the `read` system call and wait to copy the data from kernel-space memory into a buffer in the user-space of that process.

We tested 10 different payload sizes and measured the time from payload send to acknowledgement acquisition. To construct our experiment, we had two processes. The parent process forked itself at the start of execution. The child enters a loop where it waits for the parent to send data then acknowledges the request. The parent sends each payload size 10 times and calculates the minimum latency for each size. The payload sizes we used were: 4, 16, 64, 256, 1K, 4K, 16K, 64K, 256K, and 512K bytes. We ran both experiments using the `clock_gettime` and `gettimeofday` timers. We hypothesis that the majority of the latency of the Pipes will be from context switching and copying the memory in these system calls. We will test this by compiling as a static binary and using the `time` and `perf` utilities to measure time spent in user-space and kernel-space.

Results

The hypothesis that most of the time would be spent inside of the system calls was accurate to the measured results. Using the `time` utility showed that out of 0.051 seconds spent on the benchmark, 0.001 second was spent in user-space and 0.047 seconds were spent in kernel-space. To further prove this hypothesis, the `perf` utility recorded that the parent process spent 66.66% of the runtime in `__libc_read`

and 33.33% of the runtime getting the time from the clock in `__vdso_clock_gettime`.

4 Pipe Throughput Evaluation

Results

5 Conclusion